

Technical Guide & Team Onboarding

Stack, architecture, how to run, what's next · July 2026 · v1.0

Stack

LAYER	CHOICE	WHY
Framework	Next.js 16 (App Router, server actions)	Server-rendered screens, zero client API layer for v1, deploys anywhere
Database	Prisma ORM · SQLite (dev) / Postgres (deploy)	Zero-setup local dev; one-line provider switch for Vercel + Neon
Styling	Tailwind v4 + a hand-rolled design system in globals.css	The "glanceable" v4 design language: tokens, instruments, tiles, pills
Type	Fraunces (display) + IBM Plex Sans (UI), self-hosted woff2	No CDN dependency; brand carried by typography
Auth	HMAC-signed cookie sessions, scrypt password hashes	Dependency-free v1; see "before production" below

Repository layout

PATH	WHAT LIVES THERE
app/	One folder per screen (today, attendance, fees, collect, dues, my-class, diary, notices, outbox, students, settings, login) + landing page.tsx, sitemap.ts, robots.ts
app/actions.ts	All server actions (mutations): login, attendance save, payments, reminders, notices, diary
lib/	db.ts (Prisma client), auth.ts (sessions + roles), queries.ts (dashboard aggregations), messaging.ts (outbox + templates), upi.ts, format.ts
components/	shell.tsx (nav per role, Avatar, Ring, Flash), attendance-grid.tsx (client pill grid)
prisma/	schema.prisma + seed.ts (demo school: 447 students, invoices, 12 days of history)
research/ · docs/	Full market research report · this doc pack

Data model

- **School** — profile + UPI VPA (fees pay into this directly).
- **User** — role: principal | teacher | desk; teachers carry classRoomId (the class they own).
- **ClassRoom** → **Student** — students hold guardianName/guardianPhone (the WhatsApp channel) and language (en|hi, message language per family).
- **AttendanceRecord** — one row per student per UTC day, unique(studentId, date); status present | absent | leave.
- **FeeHead** → **FeeInvoice** → **Payment** — invoice status unpaid | partial | paid derived from payment totals; amounts in whole rupees.
- **Notice** — audience "all" or "class:<id>"; class diary entries are notices titled "Class diary".
- **OutboxMessage** — every parent-facing message: channel, template, rendered body, status queued | sent | failed. **All communication flows through this one table.**

Messaging architecture

lib/messaging.ts exposes `queueMessage(s)` and bilingual templates (fee_reminder, absence_alert, receipt, notice). Nothing sends directly — actions queue, a provider drains. The dev provider marks rows sent so every flow is demoable with zero credentials. Swapping in a real WhatsApp Business API provider (Interakt/Gupshup/Twilio or Meta Cloud API direct) means implementing one drain function plus webhook status updates; templates map 1:1 to WABA message templates that need Meta approval.

Auth & roles

- Session = HMAC-signed cookie carrying the user id (lib/auth.ts); passwords scrypt-hashed with per-user salt.
- `requireUser(roles?)` gates every page and action server-side; off-role users are redirected to their role home (teacher → /my-class, desk → /collect). Teachers can only save attendance for their own classRoomId.
- Demo logins: `loginAsDemo` action + one-click buttons on /login. **Remove for production.**

Before any production deployment: real session store with revocation; MFA for staff (the PowerSchool breach began with one non-MFA credential); rate limiting on login; password reset; audit log on payments; DPDP Act consent records for guardian contact data; per-school row isolation (multi-tenancy).

Run it

- **Local:** `npm install` → `echo 'DATABASE_URL="file:./dev.db"' > .env` → `npx prisma db push` → `npm run seed` → `npm run dev`. Logins:

principal|teacher|desk@sunrise.school / demo123.

- **Vercel:** switch prisma provider to postgresql, import repo, attach Neon Postgres (sets DATABASE_URL), set AUTH_SECRET, deploy, then db push + seed against the production URL. Full steps in README.

Roadmap (ordered)

#	ITEM	WHY IT'S NEXT
1	Real WABA provider + webhook delivery status	Turns the outbox real; unlocks read-tracking the design already promises
2	Board-compliant report cards (CBSE first)	The worst seasonal teacher pain; a maintained-template moat
3	UDISE+/SDMS/APAAR-shaped exports	"Enter once, export everywhere" — the compliance wedge
4	Multi-school tenancy + onboarding wizard	Self-serve growth; prerequisite for pilots beyond one school
5	Offline-first attendance (PWA + sync)	36% of schools lack reliable internet; capture must never block
6	Auth hardening (MFA, sessions, audit)	Production bar; also the sales story vs breach-tainted incumbents